



Paravia Systems

Final Project - Models of Computation

Domenico Nobile
Student ID: 22-943-039

Academic Year 2022/2023

Contents

1	Model Description	3
1.1	An informal introduction	3
1.2	Formal definition	5
2	Computing with Paravia Systems	7
3	A proof of Turing-completeness	11
3.1	Serpentine Turing Machines: definition and properties	11
3.2	Simulating a Serpentine Turing Machine with a Paravia System .	12
3.3	Simulating a Paravia System with a Turing Machine	19

1 Model Description

1.1 An informal introduction

This model is not directly inspired from anything. I came up with it by randomly thinking about some games that could be played with pen and paper, and this one immediately caught my attention, probably for its simplicity.

Imagine some ball rolling over a lattice of cells that may or may not contain some arrow. The ball follows the arrows, and *discharges* them as it passes through them. A *discharged* arrow does not deflect the ball, but gets *recharged* as soon as the ball goes through it. On the other hand, a ball rolling over an empty cell will simply preserve the direction it had at the previous step.

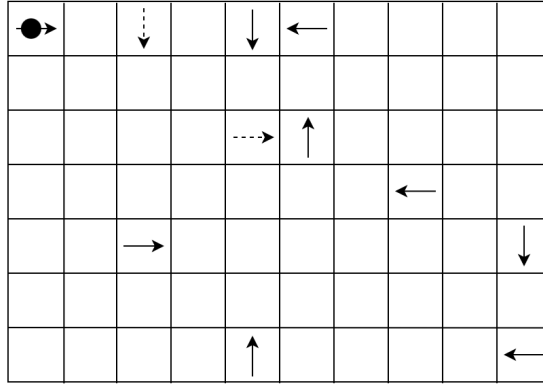


Figure 1: Notice how the two kinds of arrows (charged and discharged) differ in notation: discharged arrows are dashed.

At this point, one may ask how the ball behaves when it reaches one of the four edges of the lattice. There could be many design solutions, but for this model we will think of the lattice as a finite pattern of arrows that actually gets replicated infinitely many times in all directions. Notice that this is not a Pac-Man effect: a ball “escaping” a lattice will enter a copy of the original lattice whose arrows are independent from the previous lattice. That is, charging/discharging an arrow from a lattice does not affect any other arrow from any of the other lattices.

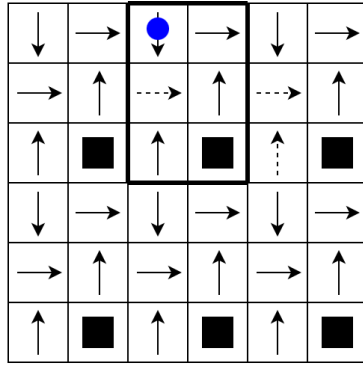
We may now assume that the ball is rolling over an infinite lattice made of arrows positioned in a periodic fashion. This makes it possible to define the infinite lattice in finite time. As part of the initial configuration of the model, the starting position of the ball gets chosen (necessarily on an arrow, to clearly define the initial direction of the ball), as well as the initial charge of the arrows.

The idea is to have the state of the arrows (charged/discharged) store infor-

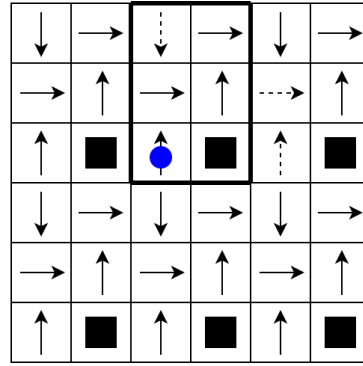
mation both as input and output, as well as memory. To allow this, it is not required for the initial state of the arrows to follow the periodic pattern of the lattice, otherwise input size would be restrained to some finite amount, upper bounded by the number of cells in the finite lattice. More in general, just as in a Turing Machine where the tape is allowed to have any initial pattern of symbols, any initial pattern of charges is allowed here. Thus, periodicity of the lattice only refers to the position of the arrows, and not to their charges at any point in time.

What remains to design is a way for the model to halt. This is simply achieved by introducing a new termination symbol (■) on the lattice. If the ball rolls over it, the system halts. As for any other symbol, the termination symbol is subject to periodicity (it will occur in every lattice in the same position).

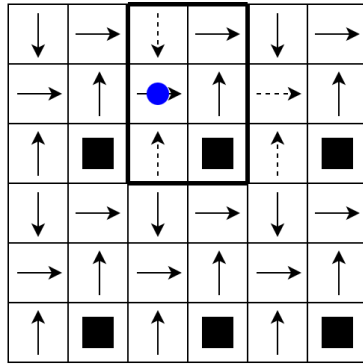
A sample simulation follows.



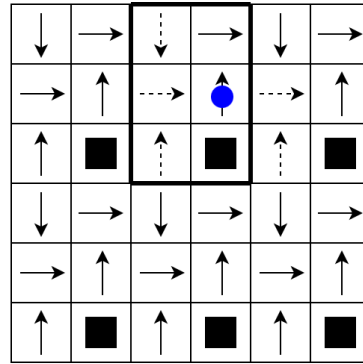
1



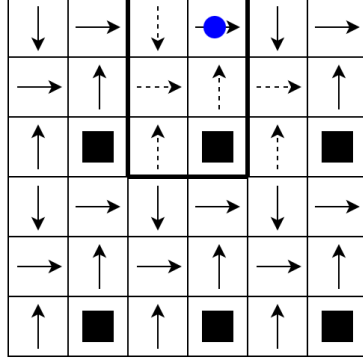
2



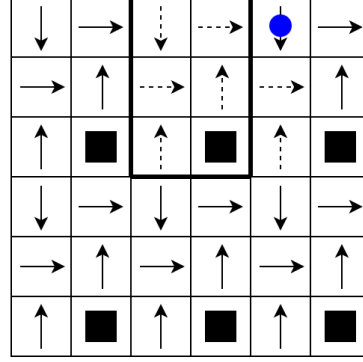
3



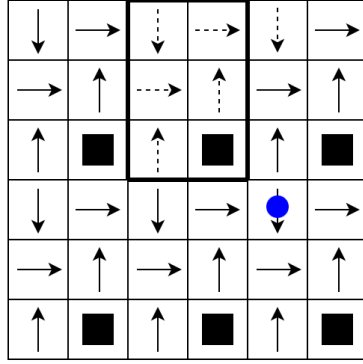
4



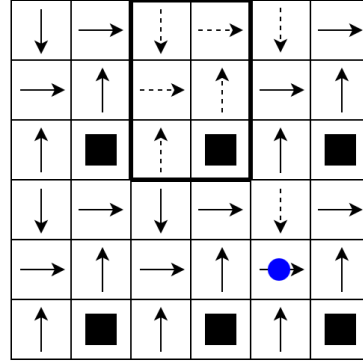
5



6



7



8

Figure 2: Not all the steps in the computation above are depicted: we only focus on those steps when the ball is on a charged arrow. Notice that the finite lattice is highlighted with a bold line containing 6 cells. The pattern of arrows gets repeated, but the initial distribution of charges is not the same for each repeated lattice. The system has not halted yet, but it could, depending on how the arrows on the infinite lattice are charged.

1.2 Formal definition

Definition 1. Let $\mathcal{A} = \{\uparrow, \leftarrow, \downarrow, \rightarrow, \square, \blacksquare\}$, $\square$ being the blank symbol and $\blacksquare$ being the termination symbol. A *Paravia System* is a triple $\langle M, x_0, y_0 \rangle$ such that:

- $M \in \mathbb{M}$ is a matrix of finite dimensions n and m of elements in $\mathcal{A}$
- $(x_0, y_0) \in \mathbb{Z}^2$ are the initial coordinates of the ball; it is required that M_{x_0, y_0} be an arrow.

We build a discrete coordinate system out of matrix M as follows: the top left cell of the matrix is at position $(0, 0)$, and the x and y coordinates grow to

the bottom right. As anticipated in the informal description, the system gets expanded in all four directions by replicating M over and over again.

At step 0, the system is at position $p_0 = (x_0, y_0)$; let p_i denote the coordinates of the ball at step i . Let $charge$ be a function that takes as input a position p and a step number i and provides the charge status of the specified cell at specified step:

$$charge_i(p) = \begin{cases} -1 & \text{if } M_p = \square, \blacksquare, \\ t.b.d. & \text{if } M_p \text{ is an arrow and } i = 0, \\ 1 - charge_{i-1}(p) & \text{if } M_p \text{ is an arrow and } p = p_{i-1}, \\ charge_{i-1}(p) & \text{otherwise} \end{cases}$$

Thus, for the function to be well defined we must first initialize the charges in every cell containing an arrow for step $i = 0$; then, for every step i we only update the charge of the cell the ball has just rolled onto.

Let $direction$ be a function of the step number recursively defined as follows:

$$direction_i = \begin{cases} (0, -1) & \text{if } M_{p_i} = \uparrow \text{ and } charge_i(p_i) = 1, \\ (-1, 0) & \text{if } M_{p_i} = \leftarrow \text{ and } charge_i(p_i) = 1, \\ (0, 1) & \text{if } M_{p_i} = \downarrow \text{ and } charge_i(p_i) = 1, \\ (1, 0) & \text{if } M_{p_i} = \rightarrow \text{ and } charge_i(p_i) = 1, \\ direction_{i-1} & \text{otherwise} \end{cases}$$

We can now clearly formalize how the ball moves. The position in the next step is defined as follows:

$$p_i = \begin{cases} (x_0, y_0) & \text{if } i = 0, \\ p_{i-1} + direction_{i-1} & \text{if } M_{p_{i-1}} \neq \blacksquare \text{ and } p_{i-1} \neq \perp \\ \perp & \text{if } p_{i-1} = \blacksquare \text{ or } p_{i-1} = \perp \end{cases}$$

This concludes the definition of a *Paravia System*. In the next section, we will explore some of the computation capabilities of this model of computation by showing some examples. In Section 3, a proof of Turing completeness is provided.

2 Computing with Paravia Systems

Computation with Paravia Systems turned out to be hard. The key idea is that we can decide the direction of the ball depending on the charge of a specific arrow by letting the ball roll onto that arrow: this will yield two distinct behaviors depending on the charge:

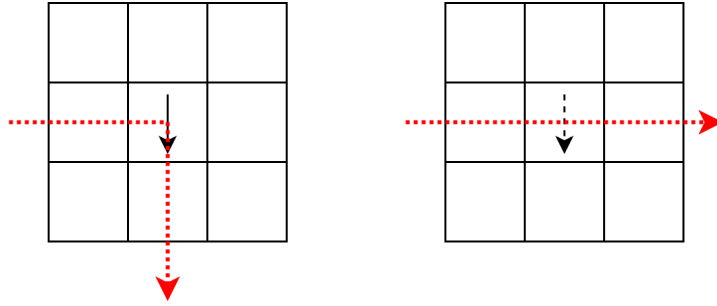


Figure 3: If the arrow is charged (left figure) the ball will flow towards the bottom of the lattice, otherwise (right figure) it will flow towards the right. Notice that separation of the behavior is only achievable when the initial direction of the ball is not that of the arrow.

To develop a deeper understanding of the model, we will build a Paravia System that recognizes a rectangle pattern. More precisely, the model will expect rows of 1s of different lengths, each row starting from the same column (Figure 4). The model will decide whether or not all rows have the same length (namely, the same number of 1s). How this input is encoded in the Paravia System is still to be specified.

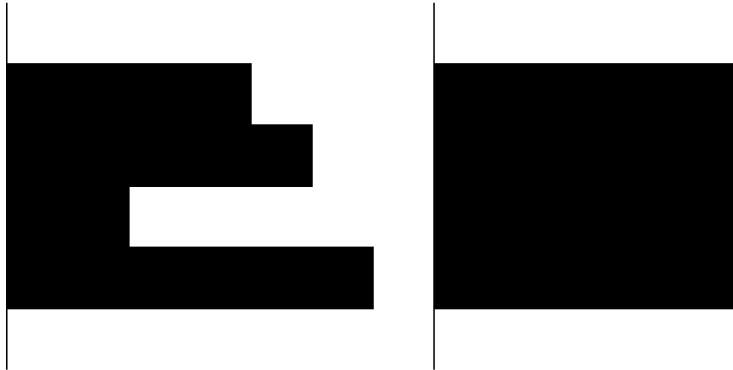


Figure 4: Left will not be accepted, right will be accepted.

...	:	:	:	:	:	:	:	:	:	...	...	:	:	:	:	:	:	:	:	...
...	0	0	0	0	0	0	0	0	0	...	...	0	0	0	0	0	0	0	0	...
...	0	0	1	1	1	1	0	0	0	...	...	0	0	1	1	1	1	0	0	...
...	0	0	1	1	1	1	1	0	0	...	...	0	0	1	1	1	1	1	0	...
...	0	0	1	1	0	0	0	0	0	...	...	0	0	1	1	1	1	1	0	...
...	0	0	1	1	1	1	1	1	0	...	...	0	0	1	1	1	1	1	0	...
...	0	0	0	0	0	0	0	0	0	...	...	0	0	0	0	0	0	0	0	...
...	:	:	:	:	:	:	:	:	:	...	...	:	:	:	:	:	:	:	:	...

Figure 5: Binary matrix representation of Figure 4

We require that the rows of 1s be left-aligned: the leftmost 1s of the rows form a vertical segment. Furthermore, rows have no holes (e.g. 111011 is not allowed) and there cannot be empty rows between non-empty rows (emptiness here means whether or not there are 1s in that row). If these preconditions are met, the model will correctly decide if the length of the rows is the same.

Before showing the actual Paravia System (*PS* from now on), it is essential to discuss how information (input, output and memory) will be handled with this model of computation. It might be tempting to construct a lattice M of arrows whose charges will be defined by some input binary matrix such that of Figure 5. However, this approach is far from solving the task: the matrix M is a fixed part of the definition of the model, whereas input may be of any size (here, number of rows and length of each row). Here is a smarter approach: M will contain some number of arrows, each with its own charge, but only one of them will be charged according to one bit of the input. That is, we no longer try to keep the whole input inside of a single matrix M ; instead, we take advantage of the infinite replication of M .

By using this approach (which will probably be clear by looking at the Figures that follow), we can finally show the model. We start by defining M :

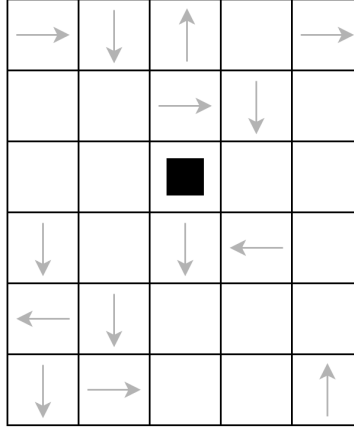


Figure 6: Notice that arrows are depicted in grey: definition of M has nothing to do with the charges of the arrows.

For the distribution of charges, here is an example of how the model is initialized to recognize the following rectangle:

	⋮	⋮	⋮	⋮	⋮	
...	0	0	0	0	0	...
...	0	1	1	1	0	...
...	0	1	1	1	0	...
...	0	0	0	0	0	...
	⋮	⋮	⋮	⋮	⋮	

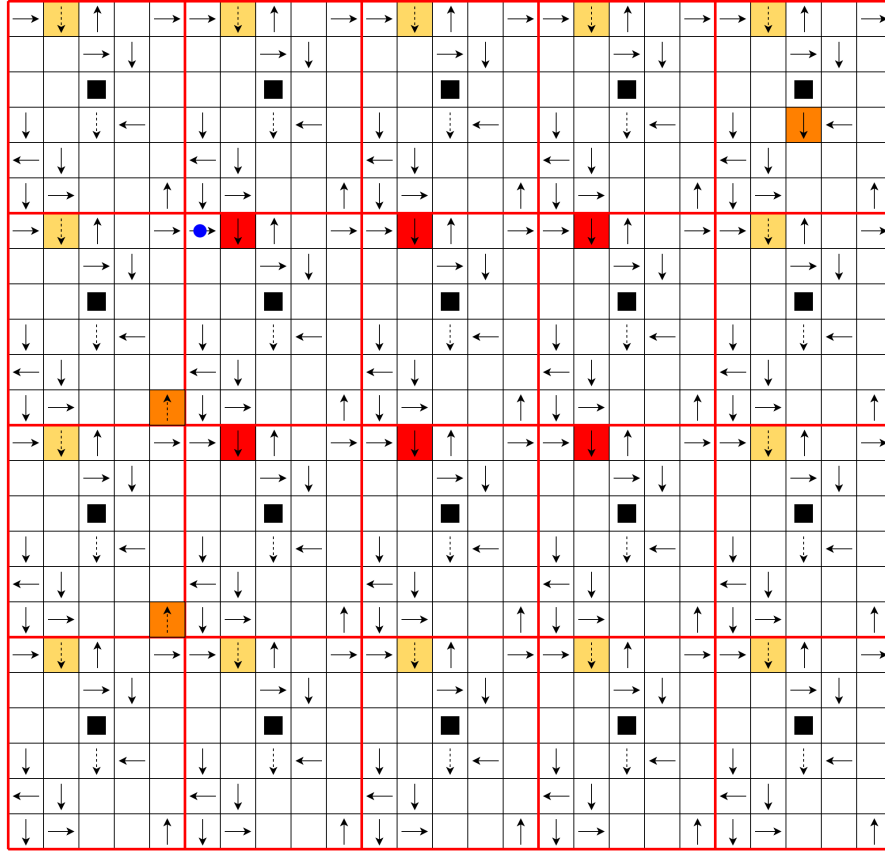


Figure 7

The figure is pretty colourful, therefore let us start from the meaning of these colors. First of all, red lines mark the border of matrix M getting replicated over and over again. For now, let us ignore the orange arrows and focus on the yellow and red ones, instead. Yellow and red arrows are the arrows containing the actual input matrix. Hence, this model basically has the actual input in the second cell of the first row of matrix M . Of course, this is not the whole model, since the lattice is infinite, but at this point it should be clear that the

rest of the arrows in that same position will be “yellow” (that is, discharged). The charges of the uncolored arrows are the same for each M . These are utility arrows necessary for the model to work. Orange arrows are utility arrows as well, but they are an exception due to their charge. More specifically, the arrow $M_{5,4}$ is always charged, except for the M s that are immediately to the left of an M containing the leftmost 1 of some row (in this case, the rows are two, therefore we have two discharged arrows in position $(5,4)$ of the finite matrix). Finally, we require the arrow $M_{3,2}$ be charged if and only if it lies on the M that is to the top right of the M containing the rightmost 1 of the first row.

This concludes the definition of the model, as we have clearly defined M and the distribution of charges while also defining how the “user” input will be fed to it. What remains to be clarified is how the output will be provided. This is done by simply looking at the charge of the arrow $M_{0,1}$, where M is the matrix right beneath the matrix containing the leftmost 1 of the last row of 1s.

The ball will be initialized on the first row as shown in Figure 7. We encourage the reader to let the ball roll by hand, as it is really space-consuming to show each step of the model on paper. The behavior is roughly this: the ball will roll over each row of 1s from left to right, and it will “mark” the first 0 at the end of the row. It will then navigate to the leftmost 1 of the next row, and do the same. Each time it reaches the end of a row of 1s, it checks whether the M right above has been marked before. If that is the case, it means that the length of the current row is the same as the length of the previous one, otherwise it will terminate. A premature termination will prevent the ball from reaching and overwriting the output cell, leaving it to 0.

3 A proof of Turing-completeness

We will show that Paravia Systems are Turing-complete by simulating a variant of a Turing machine called Serpentine Turing Machine that will be defined in a few lines. This definition was necessary to make the proof easier, although more verbose. We will first prove that this variant of a Turing Machine is actually Turing-complete, then provide a clear method of how to construct a *PS* that simulates some fixed Serpentine Turing Machine.

3.1 Serpentine Turing Machines: definition and properties

Definition 2. A Serpentine Turing Machine is a Turing Machine whose head has some constraints on how it can move: it will keep on moving in the same direction (either left or right) until exactly two new cells of memory have been visited. Then, it must change its direction and keep on moving with the same behavior.

Assuming that the initial direction is to the right (**R**), we can already define the sequence of movements of the head:

R L L L R R R R L L L L L L L . . .

In other words, the tape is visited by moving the head back and forth, and each time it will span a two cells larger portion of memory. This guarantees a property that is essential for the *PS* that will be shown in a few lines to properly work: at any point in time (that is, at any step of the computation) each memory cell will have been visited a number of time that differs of at most 2 from the number of times its two neighboring cells have been visited.

Before moving on to the *PS*, let us quickly discuss why this restriction on the movement of the head does not affect the capabilities of the Turing Machine. Of course, if the forced movement of the head is the same as intended by the ordinary Turing Machine, there is nothing to show. Now, suppose that the ordinary Turing Machine wants to move the head to the opposite direction. We can simulate this movement in a Serpentine Turing Machine by transitioning to a temporary state that signals that whatever is going to be read from memory should be ignored until the machine comes back to the same spot on memory. At this point, the direction of the Serpentine Turing Machine will be compatible with the direction intended by the ordinary Turing Machine. We should mark the symbol where this problem arose, so that the machine will know when to exit this temporary state. What if the Serpentine Turing Machine has to simulate a non-moving head? To do this, there will be a second temporary state. The machine will then mark the symbol right next to the symbol where it should have kept the head. When the head of the Serpentine Turing Machine will have come back to the marked symbol, it will know that in the next cell it will perform a step of the ordinary Turing Machine. What remains to treat is how the machine decides to invert the direction of its head. To do this, we can add

to each symbol in the alphabet a subscript L or R , denoting the current left and right borders of the explored memory. These will be moved further away by 2 cells each time the machine reaches them. Note that we can add as many symbols as we want: a proper binary encoding will reduce the STM alphabet to 0s and 1s (at the cost of additional states) without breaking its behavior.

3.2 Simulating a Serpentine Turing Machine with a Paravia System

To design a PS that simulates a Serpentine Turing Machine, we will use a similar approach as in Section 2 for input encoding: we will assign the value of a cell of the memory tape to one single specific arrow on M . This will be done for each of the infinite M s *lying on the same horizontal line*, ignoring the M s on any other line. This gives a clear, geometric view of the simulated Turing Machine.

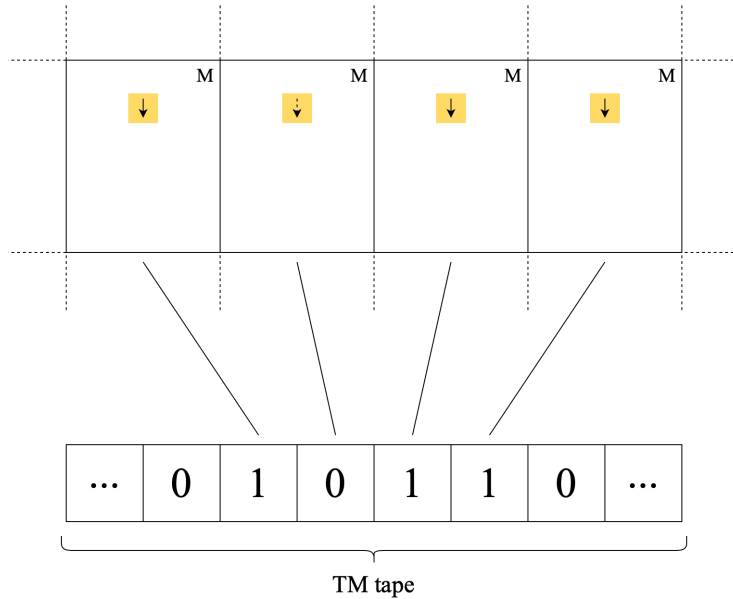


Figure 8

However, this nice view of the TM will soon be destroyed. The idea is to let the ball roll on the yellow arrows in Figure 8 and choose where to go (either left, right or nothing) based on the current state of the simulated machine. How state will be encoded in the PS will become clear later. Before that, it is essential to discuss the reason why Serpentine Turing Machines have been introduced in the first place. The problem with Paravia Systems is that a set of arrows has some behavior when used the first time, that gets then altered due to changes in

the charges of some arrows. However, it would be nice to reset in some way the arrows, so that the ball will be sent on the correct directions. However, doing this can be really painful, and the suspect is that it is not even possible for many sets of arrows. This is a problem for the simulation of a Turing Machine, at least in this nice initialization of the *PS* (Figure 8), as we would like to keep the behavior of the ball when it enters a previously visited *M* unaltered.

To avoid this problem, the idea is to make sure that each *M* is visited at most a fixed, finite number of times. This is not reasonable when we are working with Turing Machines, as their behavior is unpredictable. Thus, we will slightly modify the original idea as follows: each time the ball visits some *M* (recall that each *M* is associated to a cell of the Turing Machine tape), it will write whatever the Turing Machine decides to write *on the M right above it*. The next time the ball will need to read that cell, it will climb up to the first unused *M* in this vertical stack of *Ms*.

The reason why Serpentine Turing Machines come in useful here is that the number of *Ms* that get climbed is always at most 2. This is due to the property of Serpentine Turing Machines mentioned earlier: at each step each cell of the tape is visited a number of times that differs no more than 2 from its two neighboring cells. This will make the design of the climbing system much easier. In practice, what will happen is that the stacks will grow more and more, forming a sort of pyramid centered in the first *M* (see the sequence of head movements in Definition 2).

Let us now proceed to show how the Serpentine Turing Machine will actually be implemented with a *PS*. For practical purposes, we will focus on a three state Turing Machine, as Serpentine Turing Machines typically have too many additional states. However, Serpentine Turing Machines are a subclass of Turing Machines, and since what we are about to design can be easily tweaked to have as many states as we want, programmed to do anything we want, this will not be a problem. However, it is worth to note that what is about to be shown will not work, as the simulated machine is not a Serpentine Turing Machine. Here is the *TM*:

Current State	Read 0			Read 1		
	Write	Move	Next	Write	Move	Next
→A	1	R	A	0	R	C
B	1	L	B	1	N.D.	H
C	0	L	A	0	L	B

Figure 9: Turing Machine Transitions. *A* is the initial state. *H* denotes the halting state.

We can now proceed to define the *PS* that will simulate the *TM*. The claim is that this *PS* will correctly simulate it *if the simulated TM is a Serpentine Turing Machine*.

First of all, consider the set of arrows in Figure 10.

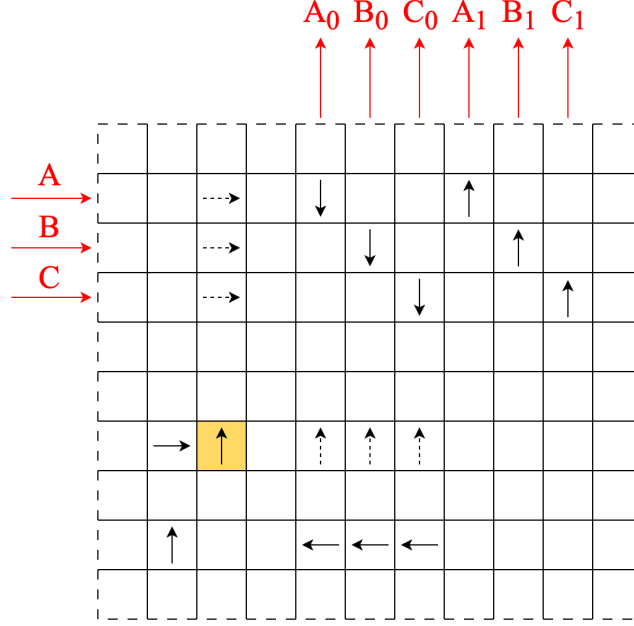


Figure 10

We will call this box of arrows a *Memory Unit*. Basically, the ball will enter it from 3 possible lanes (highlighted as red arrows in Figure), and it will get out of the box from 6 possible exits (again, highlighted as red arrows in Figure). Suppose the ball enters the *Memory Unit* from entry X . It will then get out of the box at exit X_b , b being the charge of the arrow highlighted in yellow, that is, the arrow associated to a memory cell on the tape of the simulated *STM*. Hence, a *Memory Unit* combines the information of the state the simulated *STM* is in (encoded as the input lane) with the bit read from memory. We can now decide what should be done based on state and memory, just like in a *TM*. More specifically, the model should:

1. decide whether memory should be overwritten or not
2. decide the next state
3. decide the direction of movement of the head (decide the next M , in *PS* terms), either right or left, since we are simulating an *STM* instead of a *TM*.

To achieve this, we will use the *Transition Control Unit* in Figure 11. This is the core of the simulation: changing the position of the arrows inside this unit is like tweaking the table in Figure 9, and the reader is encouraged to see why the unit depicted below is exactly the same as the table above.

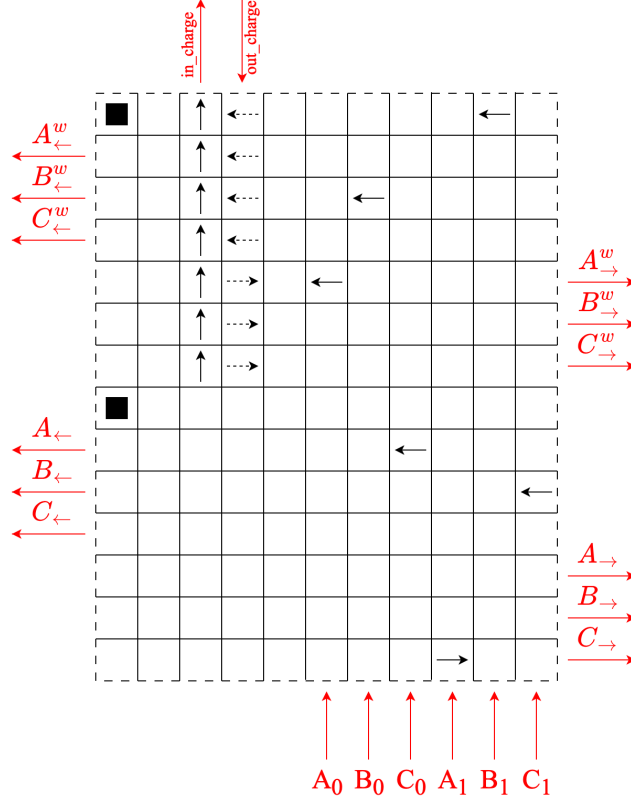


Figure 11: A *Transition Control Unit* modeling the *TM* in Figure 9. Notice that not all the rows are necessary, and could be removed without affecting the behavior of the *PS*. All rows have been kept for clarity.

A crucial component of the *Transition Control Unit* is the set of arrows on the upper left half of the unit. Recall that this *M* will be used only once, and the memory information inside the *Memory Unit* should be transferred to the *M* directly above the current *M*. To keep things simple, the yellow arrow in the *Memory Unit* will be initialized to 0 on the rows of *Ms* above the row of *Ms* containing the initial tape of the simulated *STM*. Hence, point 1 in the to-do list above may be rephrased as “decide whether to write a 1 on the *M* above or keep it at 0”. This clarifies why there are arrows going up: the ball is temporarily sent to the *M* above to write a 1. To do this, we should quickly add some arrows to the *Memory Unit* in order to handle this problem, just like in Figure 12.

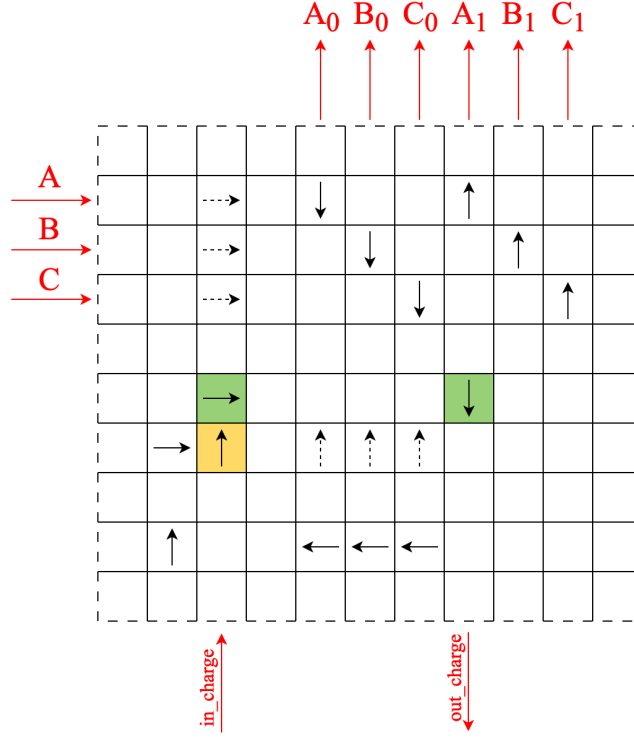


Figure 12: New version of the *Memory Unit*, now with memory prewriting capability. Green arrows are the newly added ones. They will be used only when the yellow arrow is discharged and the ball comes from the *in_charge* lane. This will charge the yellow arrow and discharge the green arrows, making it equivalent to the *Memory Unit* of Figure 10.

With this, we have correctly achieved points 1 and 2 in the to-do list. What remains to do is find a clever way of moving to the next M , keeping in mind that M s stack.

The idea is the following: suppose the head is moving to the right; the ball will exit the current M and navigate through the M on the bottom right of the current one. If that M has already been used, it will check for the M right above it. Again, if already used, it will go above of one additional M . The pattern produced by *STMs* guarantees that one of the three M s will be unused at each step. Moreover, each M will be checked for availability at most three times, and in each of these three times the checked M will be in a distinct height relative to the current M of the ball (either below, same height, or above, in this exact order, making it sure that the M above is certainly available). Hence, we can build in each M two distinct sets of arrows that check whether that M has been used or not, and make sure that each of them will be used at most

once, guaranteeing the intended behavior of the subsystem. To implement this, a final tweak to the *Memory Unit* is necessary (Figure 13).

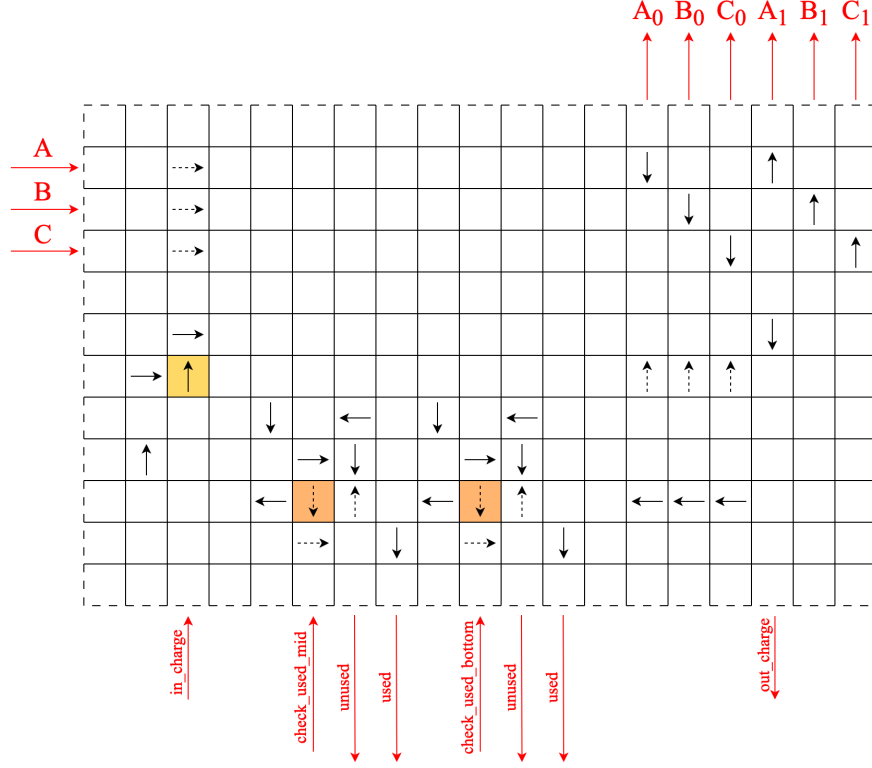


Figure 13: The final version of the *Memory Unit*. Orange arrows highlight the two “sensors” that will stop the ball from working with this M if it has been used previously. The reason why there are two distinct sensors is that this check will be performed at most three times, in three distinct situations (M is below, above or at the same height of the previous M), but when M is above we do not require any checks, as M will be certainly available due to the behavior of *STMs*. Having a separate sensor for each situation allows us to construct a “sensor system” that should not be used more than once, making it easier to design the *Memory Unit*.

Finally, we can put everything together to define the complete M of the Paravia System (Figure 14).

“sensor” arrows belonging to the real *Memory Unit*, acting as sensor that tells if the M has been used. If the check happens on the rightmost arrow and the M has been used, the ball gets sent to the M above, where the same check will be performed, but on the second sensor instead of the first. If this M has been used too, the ball will navigate and directly use the M above, which is certainly available.

Note that the model is clearly scalable to any number of states. This can be easily achieved by adding a proper amount of additional lanes in the *Memory units*, in the *Transition control unit* and in the other “buses” spread around these main components.

To properly define the PS , as always, the distribution of charges should be clearly defined. For this specific model, charges are mainly periodic, except for:

- the yellow arrow of the *Memory Unit*, which actually contains the real input to the model on the first row of M s, and 0 everywhere else;
- the sensors on the row of M s right below the row of M s containing the input: these sensors should be activated, to trick the model into thinking that those M s have previously been used;
- the arrow right above the yellow arrow: if the yellow arrow is charged, this should be discharged, otherwise it is charged.

Finally, it should be clarified where the output of the simulated STM should be looked for: the correct output tape should be read by picking the M right above the highest used M in each stack. Due to the serpentine movements of the STM , the pattern will be similar to a pyramid centered on the first cell read by the STM .

This concludes the proof. Again, the key idea is that the PS correctly simulates a specific subclass of Turing Machines, the Serpentine Turing Machines defined above. Since this subclass still manages to keep the computational capabilities of ordinary Turing Machines, we can conclude that Paravia Systems are Turing-Complete. In the last subsection, we will explore how Turing Machines can simulate a Paravia System, proving that the two models are actually equivalent.

3.3 Simulating a Paravia System with a Turing Machine

This is a way easier task. More specifically, we will prove how to simulate a Paravia System with a Turing Machine having a bidimensional tape. The Machine tape will have three possible symbols: 0, 1 and * for “blank”. The tape will be initialized resembling the infinite lattice of the Paravia System, with a 1 if the arrow is charged, 0 if it is discharged and * whenever there is no arrow on that cell. Furthermore, there will be 4 states for each cell in M : the states encode two things: the direction of the potential arrow lying on a cell of M and the current direction of the ball (this is why there are actually 4 states for each M cell, one for each current direction of the ball). In each state,

reading a $*$ never overwrites memory, whereas reading a 0 or a 1 always flips the bit read, to simulate the dynamics of the charges. Recall that each state carries both the current direction of the ball and the current position in M . Each state decides the next state and the head movement as follows:

- if the current state reads a $*$ or a 0, the next state is decided by the current direction of the ball, otherwise it is decided by the value of the charge. A way to imagine this is to construct a graph of states and embed it in the space. A state encoding a right arrow will be connected to a the state to its right, moving on the x-axis; if it is the rightmost state, the next state will then be the leftmost one (Pac-Man). If the simulated ball has changed direction, the next state will be to the right of the current state, but will also change its depth (z-axis), since there are 4 possible depths, one for each current direction of the ball.
- The head movement is the same as the movement in the graph of states, ignoring the Pac-Man effect. That is, if the ball is on the right edge of M and wants to move to the right, the tape will still be moved to the right.

There is a clear connection between the components of a Paravia System and those of a Turing Machine: the tape is the distribution of charges, while the states encode the configuration of M and the movement of the ball. Whenever a termination cell is reached, the Turing Machine will reach a halting state, and will terminate, yielding exactly the same output as the simulated Paravia System.